INDEX

NAME AERIL BIVISHA.B SUBJECT CN (Observation)

STD. DIV. ROLL NO. 240701021 SCHOOL

SR. NO.	PAGE NO	TITLE	DATE	TEACHER'S SIGN / REMARKS
1	1	STUDY OF NETWORK COMMANDS	26/07	27/7/26 (9)
2.	8	STUDY OF DIFFERENT TYPES OF NETWORK	02/08	4/8/26 (9)
3.	10	CISCO PACKET TRACER(SIMULATION TOOL)	11/08	18/8/26 (9)
4.	13	Setup and configure A LAN LOCAL AREA using a switch and Ethernet cables in your lab	11/08	11/9/26 (9)
5.	16	Packet capture tool: Wireshark	13/09	14/9/26 (9)
6.	20	Error Correction at Data link layer	14/09	18/9/26 (9)
7.	23	Flow Control at Data Link layer	16/09	19/9/26 (9)
8.	26	Nmap to Discover live Hosts using Nmap on the TryHackMe Platform	19/09	22/9/26 (9)
9	29	Implementation of Subnetting in cisco packet Tracer Simulation	24/09	25/9/26 (9)
10	31	End-End Communication at Transport layer	25/09	26/9/26 (10)
13	33	Implement your own ping Request	26/09	29/9/26 (10)
14.	35	Write a code using RAW sockets	29/09	29/9/26 (10)

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
10	36	a) Interworking with source within Cisco packet tracer Simulator b) Design and configures an inter network using wireless router, DHCP server and internet cloud	30/09	9 30/9
11	39	a) Simulate static routing configuration using Cisco packet tracer b) Simulate using Cisco packet tracing	30/09	10 30/9

Ex: No:1
DATE:
26/07/24

STUDY OF NETWORK COMMANDS

Windows Commands

1) arp-a : ARP is short form of address resolution protocol

O/P

Interface : 10.16.56.50 ... 0x9

Internet address	physical address	Type
10.16.56.155	ee-12-17-L7-12-5L	
10.16.56.255	ff-ff-ff-ff-ff-ff	

2) hostname : This is the simplest of all TCP/IP command. It will display the name of your computer

O/P: LAPTOP-TOO3BCTN

3) ipconfig /all : This command displays configurat information about your TCP/IP connection.

O/P:

Window IP configuration

Host Name . . . . . : LAPTOP-TOO3BCTN

Mode Type . . . . . : Mixed

4) nbstat -a : This command helps solve problems with netBIOS name resolution

O/P

Ethernet 2:

Mode IP Address : [192.168.56.1] Scope Id: []

Host not found.

5. netstat: (Network statics) netstat display a variety of statistics about a computer Active TCP/IP connection.

O/P: Active connection

Proto	local address	Foreign Address	State
TCP	10.16.56.50:4944	+.213.25.242	Established

6. nslookup: It is used to display the DNS such as IP address of a particular computer

O/P:

Server: unknown

Address: 10.16.56.55

7. pathping: Pathping is a unique to windows and its basically a combination of the ping and Tracert commands.

cmd: pathping -g google.com

O/P

Tracing route to google.com [142.250.66.14] over a maximum of 30hops.

Trace complete.

8. ping: ping command is the but way to test connectivity between two nodes.

ping google.com

O/P:

pingping

9. Route: route command is used to show/manipulate the IP routing table. It is primarily used to static routes to specific host or network

via an interface
: Manipulate network routing table

Route: [-f] [-p] [-4] [-6] command [destination]

2nd | Route

13... 0a 00 27 00 00 0d ... VirtualBox Host-only Internet Adapter

Linux Commands.

1. IP commands.

1. cmd: ip address show - Displays all IP address of networks interfaces.

O/P: eth0: inet 192.168.1.10/24

2. ip address add 192.168.1.254/24 dev enp0s3 - Assigns an ip address to an interface.

O/P: IP address added successfully

3. ip link set eth0 up - Enables the network interface

O/P: eth0 state up

4. ip link set eth0 down - Disables the network interface.

O/P: eth0 state down.

5. ip link set eth0 promic on - Enables promiscuous mode

O/P: Promiscuous mode enabled.

6. IP route add default via 192.168.1.254 dev eth0 - Adds the default gateway.

7. Ip route get 10.10.1.4 - shows the route to a destination IP.

O/P: 10.10.1.4 via 192.168.1.254 dev eth0.

2 if config

Displays network interface configuration

O/P:

eth0 link encap: Ethernet inet 192.168.1.10

3. mtr

1. mtr google.com - shows route statistics to a host

O/P:

HOST: localhost loss % avg Bet recv

2. mtr -g google.com - shows numeric IP address only.

O/P:

142.250.183.206.

3. mtr -b google.com - shows both hostname and IP address

O/P:

google.com(142.250.183.206)

4. mtr -c 10 google.com - sends only 10 packets.

O/P:

sent 10 packets.

3. tcpdump.

1. dnf install -g tcpdump - install tcpdump

2. tcpdump -D. - list available capture interfaces

O/P:

1. eth0 2. lo

3. tcpdump -i eth0 -c 10 - capture packets from a host.

O/P:

10 packets captured.

4. tcpdump -i eth0 src host 8.8.8.8 - captures packets from a host

O/P:
8.8.8.8 > 192.168.1.10

5. tcpdump -i eth0 dst host 8.8.8.8 - capture packets to a host

O/P:
192.168.1.10 > 8.8.8

6. tcpdump -i eth0 port 53 - captures DNS traffic

O/P: DNS query captured

7. tcpdump -i eth0 -c 10 host www.google.com and port 443 - captures HTTPS packets

O/P: TCP 443 captured.

8. tcpdump -i eth0 port not 53 and not 25 - captures all traffic except DNS and SMTP

O/P: TCP/IP packets displayed.

9. ping:

1. ping google.com - checks connectivity to a Host

O/P:
64 bytes from google.com time = 12ms

2. ping -c 10 google.com - sends only 10 ping request

O/P:
10 packets transmitted - 10 received

nmcli

1. nmcli connection show - lists all network connections.

O/P: wired connection

2. nmcli connection ~~show~~ modify "wired connection" Modifies an existing connection.

3. nmcli connection show - Displays connection details.

O/P: ipv4.method auto.

4. nmcli connection modify "wired connection" ipv4.method auto - configures DHCP

O/P: IPV4 method set to auto.

5. nmcli connection modify "wired connection" ipv4.method manual ipv4.address 192.0.2.1/24 ipv4.gateway 192.0.2.254 ipv4.dns 192.0.2.200 - configures a static IPV4 address

O/P: IPV4 setting updated

6. nmcli connection modify "wired connection" ipv6.method auto - configures automatic IPV6.

O/P: IPV6 method set to auto

7. nmcli connection up Internal-LAN: Active the connection profile.

Result:

The above both the windows and Linux commands are executed and outputs are completed successfully. 27/7/26

Ex. No: 2
03/08/24

STUDY OF DIFFERENT TYPES OF NETWORK

CABLES

AIM:

a) Understand different types of network cable

Different type of cables used in networking are:

1. Unshielded Twisted Pair (UTP) Cable
2. Shielded Twisted Pair (STP) cable
3. Coaxial cable
4. Fibre optic cable

Cable Type	Category	Maximum Data Transmission	Advantages/ Disadvantages	Application/ use
UTP	Category 3	10bps	Advantage: *Cheaper in cost * easy to install as they have a smaller overall diameter Disadvantage: * More prone to Electromagnetic interference and noise	NO Basic-7 Ethernet
	category 5	upto 100 mbps		Fast Ethernet Gigabit Ethernet
	category 5e	1 Gbps		Fast Ethernet Gigabit Ethernet
STP	category 6a	10GBPS	Advantage: * Shielded * Faster than UTP * Less to noise and interference Disadvantage *Expensive * Greater Installation effort	Gigabit Ethernet 10G Ethernet (upto 100m) widely used in data centre
	category 7	10Gbps		Gigabit Ethernet 10G Ethernet (100m)

Cable type	Category	Maximum Data Transmission	Advantages / Disadvantages	Application / Use
Coaxial cable	RG-6 RG-59 RG-11	10 - 100 Mbps	Advantages: * High bandwidth * Immune to interference * Low loss bandwidth * volatile Disadvantages * Limited distance * Cost * size is bulky	Speed of signal is soon Television network High speed network connections.
Fibre optics-cable	Single mode, Multi mode	1000 Gbps	Advantages: * High speed * High bandwidth * High security * Long distance Disadvantage * Expensive * Requires skilled installers	Maximum distance of fibre optic cable is around 100 m

RESULT

4/6/26

The above different types of cables are explained successfully

Ex. No:3
9.8.24

CISCO PACKET TRACER (Simulation Tool)

AIM:

To study the installation and user Interface of Cisco Packet Tracer and understand the behavior of network device such as Hub and Switch which forwarding products

INTRODUCTION:

* Cisco Packet Tracer is a network simulation and modeling tool used to design and computer networks.

* It allows users to create network topology configure devices and observe packet flow without requiring physical networking equipment.

INSTALLATION OF PACKET TRACER

* Cisco packet Tracer can be downloaded from the Cisco Networking Academy website.

* After downloading the appropriate version for the operating system, the installation can be completed by following the setup instructions.

USER INTERFACE OVERVIEW:

1. Menu Bar - Provides options.
2. Main Tool bar - contains commonly used tools.

3. Logical / physical workspace Tabs - used to with logical and physical view.

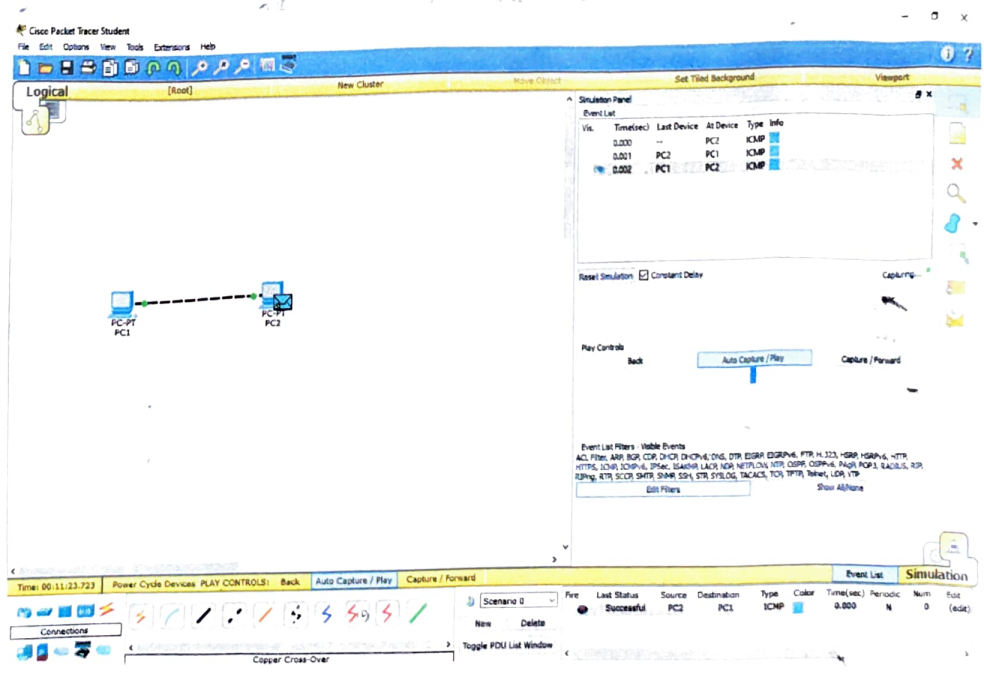
4. workspace - Area where Network topologies are created.

5. Common Tool Bar - Provides tools for moving etc.

6. WSIT - Acard Product Box - used to create customized products.

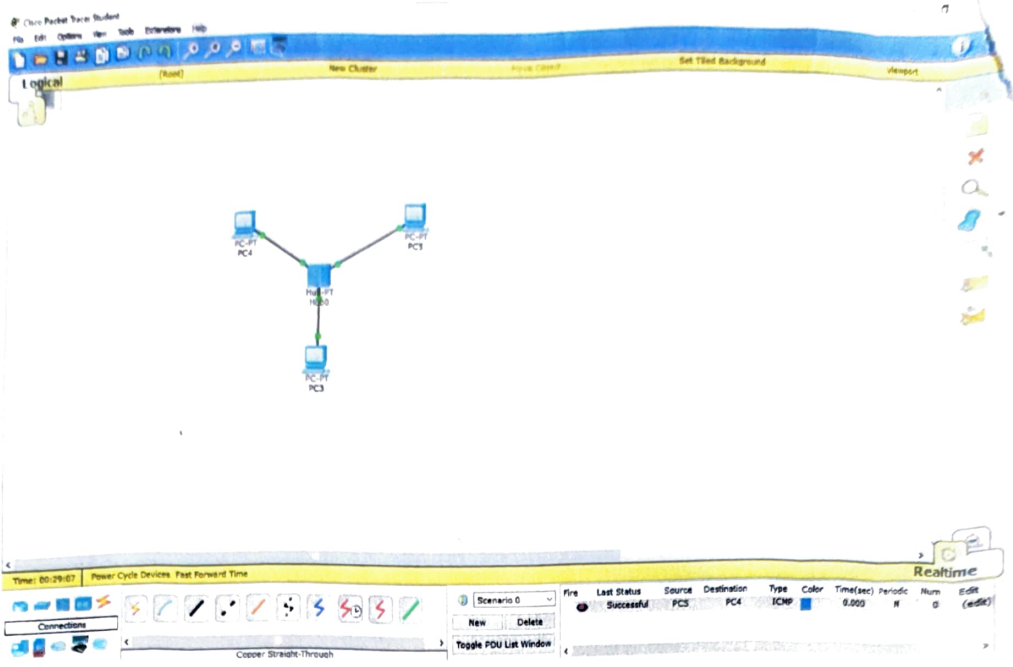
Analysis of Networks Devices using Cisco Packet Tracer

Peer - to - Peer



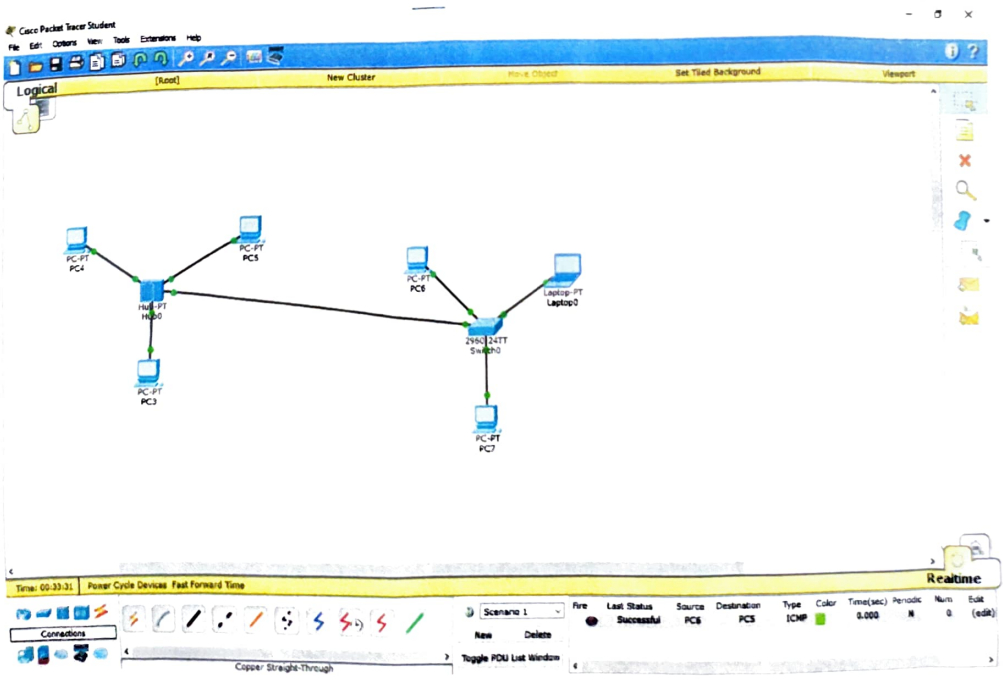
- Place two PC's and connect them using copper Cross over Cable
- Configure IP address for both PC's in the same network
- Use Ping to verify the connection

Hub Network



- Place the PCs and a Hub, Then connect them using copper straight through cables
- Use Ping to verify the communicatio

Switch Network



- Place a switch and connect with the hub and other PCs using copper straight - through cables.
- Configure IP address for the connected devices.

RESULT

Cisco Packet Tracer was successfully studied and used to create a simple network.

SETUP AND CONFIGURE A LAN LOCAL AREA NETWORK USING A SWITCH AND ETHERNET IN YOUR LAB

AIM

To setup and configure a LAN (Local Area Network) using a switch and Ethernet cables in Cisco Packet Tracer and verify the connectivity between the devices.

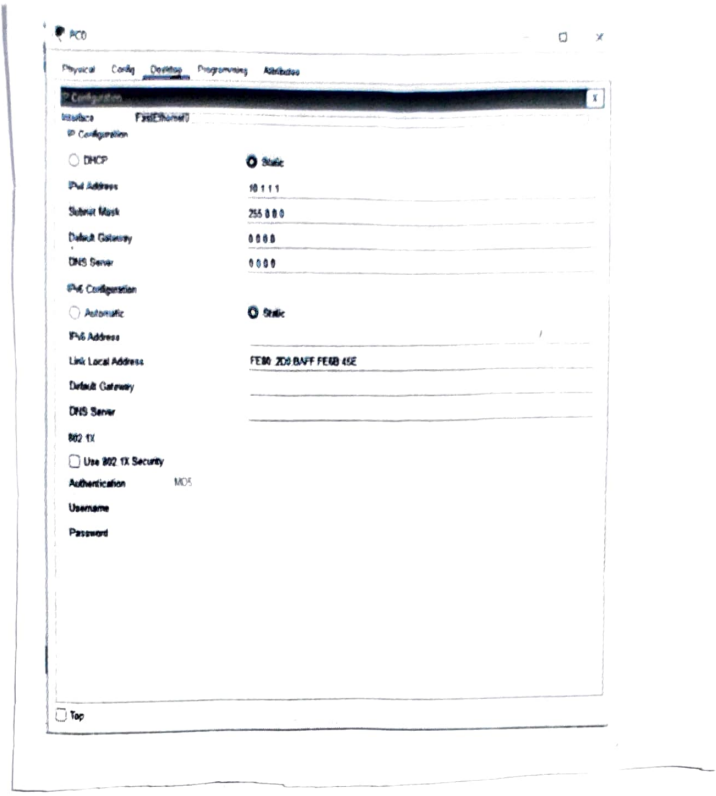
PROCEDURE

STEP 1: Open Cisco Packet Tracer and create a LAN Topology using PCs and 1 Switch

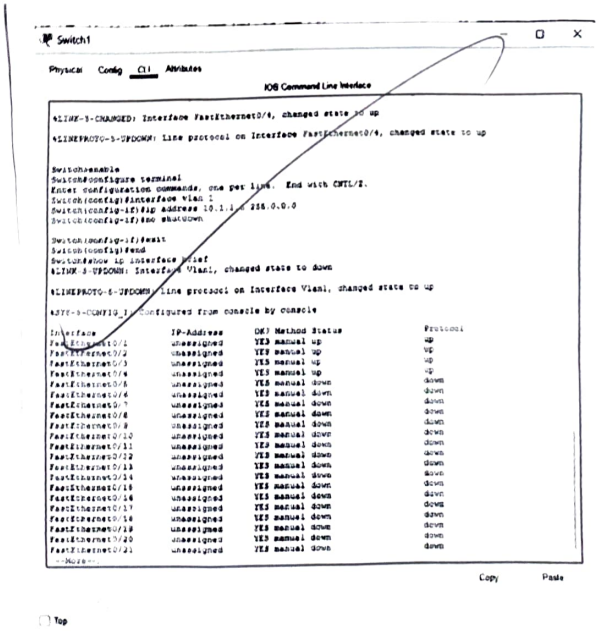
STEP 2: Connect all the PCs to the switch using copper straight-through Ethernet cables.

STEP 3: Configure the IP address for each PC using the following settings:

Device	IP address	Subnet Mask
PC0	10.1.1.1	255.0.0.0
PC1	10.1.1.2	255.0.0.0
PC2	10.1.1.3	255.0.0.0
PC3	10.1.1.4	255.0.0.0



STEP 4:.. Configure the switch using the CLI Assign the switch VLAN 1 interface the IP address 10.1.1.5 with subnet mask 255.0.0.0 and enable the interface using the no shutdown command

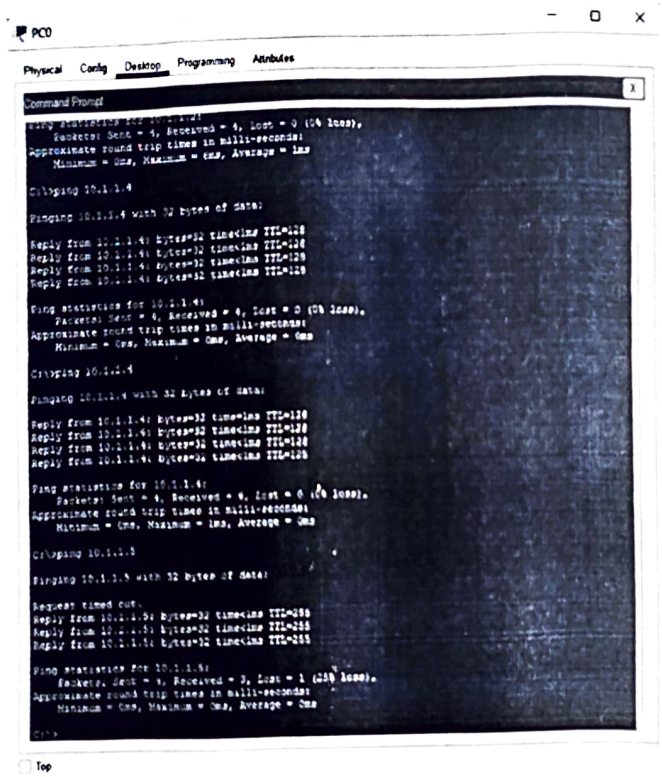


STEP 5: Verify the connectivity between the PCs and switch using the ping command from the command Prompt

Ex:

Ping 10.1.1.4

Ping 10.1.1.5



RESULT:

Thus, a LAN was successfully configured using a switch and Ethernet cables in Cisco Packet Tracer, and the connectivity between the

Fri.
Date: 15/09

PACKET CAPTURE TOOL : WIRESHARK

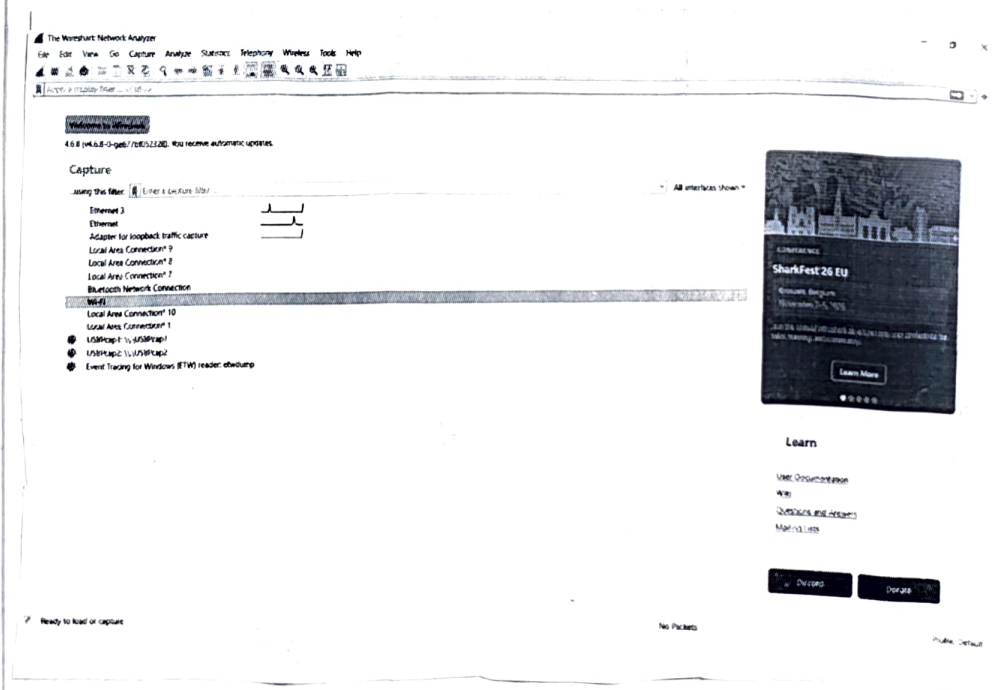
AIM:

To understand the features of wireshark as a packet capture tool and understand encapsulation of information of various layers of a Protocol shark.

PROCEDURE:

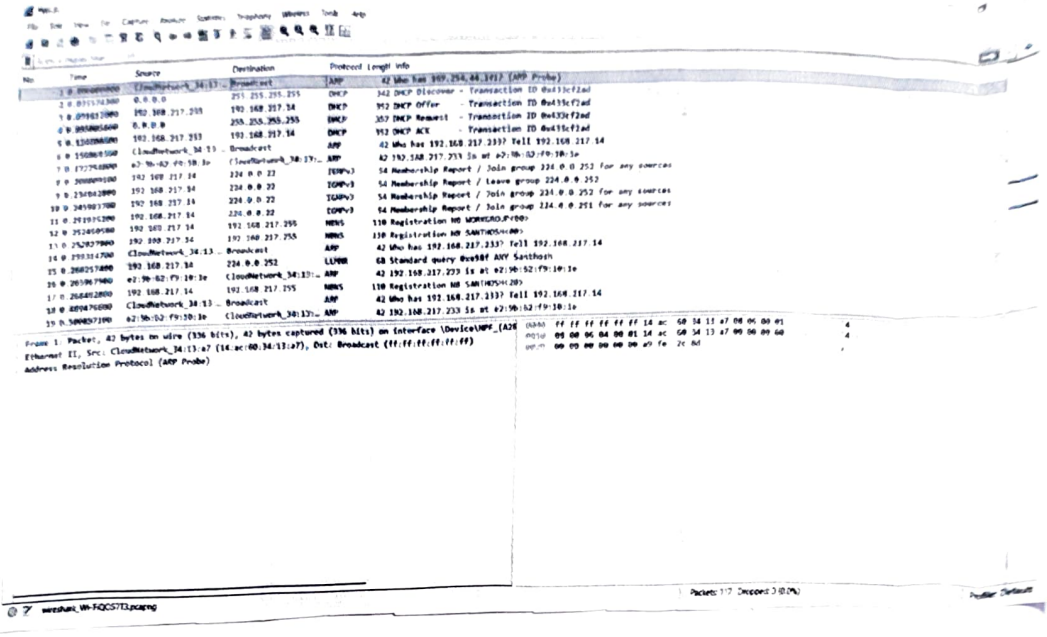
STEP 1: Open Wireshark and select the wifi network interface.

STEP 2: Start the packet capture by double clicking the wifi interface.



STEP 3: observe the packet being captured in the wireshark window.

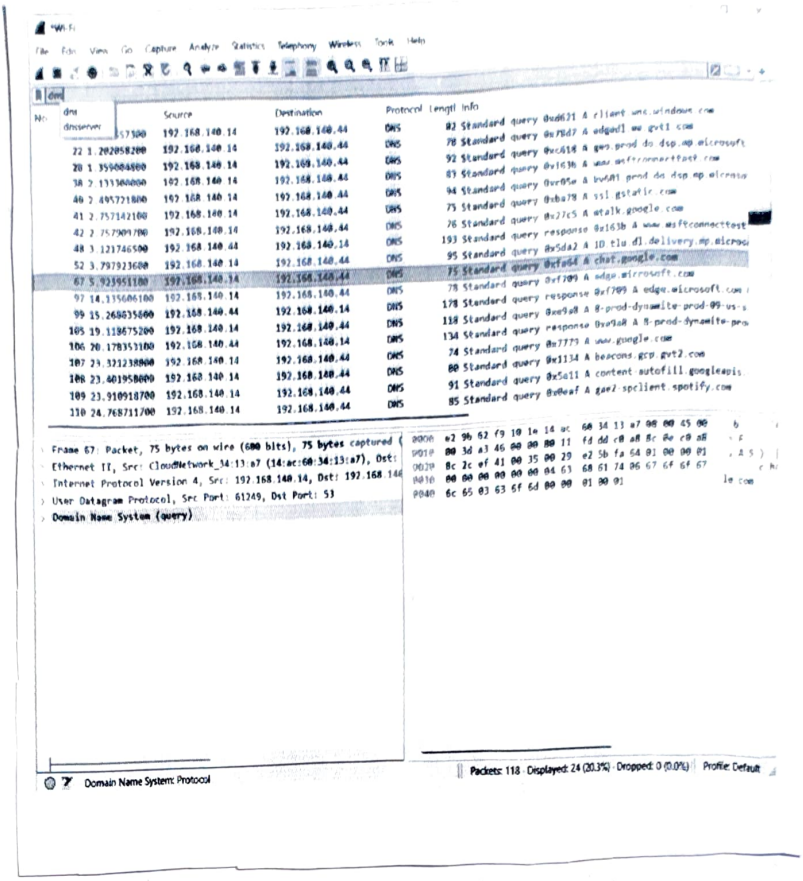
STEP 4: Different packets such as ARP, DHCP, ICMPv6, MDNS and LLMNR can be observed in the Packet list



STEP 5: Select any packet to view, its detailed implementation such as Frame, Ethernet II, IP, UDP/TCP and protocol details.

STEP 6: Apply the display filter in the filter bar to display only DNS packets.

STEP 7: Observe DNS Query and packets. The source and destination IP address, protocol, packet length and DNS information can be analyzed.



STEP 8:

Stop the packet capture after collecting the required packets.

RESULT:

Thus, network packets were successfully captured and analyzed using wireshark and DNS packets were identified using the display filter

13/9/24

ERROR CORRECTION AT DATA LINK LAYER

AIM:

Write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data and verify over correction feature

Code:

```
data = input("Enter a data bits: ")
d1, d2, d3, d4 = map(int, data)
P1 = d1^d2^d4
P2 = d1^d3^d4
P4 = d2^d3^d4
code = [P1, P2, P1, P4, d2, d3, d4]
print("Hamming code: ", "".join(map(str, code)))
received = list(map(int, input("Enter received code")))
```

S1 = received[0]^ received[2]^ received[4]^ received[6]

S2 = received[1]^ received[2]^ received[3]^ received[6]

S4 = received[3]^ received[4]^ received[5]^ received[6]

```
error = S4*4 + S2*2 + S1
if error == 0:
    print("No error")
```

```
else:
    print(" Error at position :", error)
    received [error-1]^=1
    print("corrected code :", ".join(map(str,
received)))
```

O/P

```
Enter 4 data bits : 1001
Hamming code : 0011001
Enter received code : 1001011
Error at position : 4
corrected code : 1000011
```

RESULT:

Thus, the Hamming code was successfully generated the error was detected, and the erroneous bit was correctly placed.

14/9/26

FLOW CONTROL AT DATA LINK LAYER

AIM:

Write a program to implement flow control at data link layer using sliding window protocol, Simulate the flow of frames from one node to another

CODE:

sender.py

```
message = input("Enter message:")
window = int(input("Enter window size:"))
i=0
while i<len(message):
    print("\n Sender Buffer:")
    for j in range(i,min(i+window,len(message))):
        print("frame",j+1,":",message[j])
    ack = int(input("Enter Ack received:"))
    if ack>=i+1 and ack<=min(i+window, len(message)):
        print("ACK accepted")
        i=ack
    else:
        print("Wrong ACK - Resending frames")
print("\n All frames sent successfully")
```

receiver.py

```
frame = int(input("Enter received frame number:"))
data = input("Enter frame data:")
expected = int(input("Enter expected frame
```

```
print("In Receiver:")
    frame = accepted
print("Frame", frame, "received")
print("Data", data)
print("ACK", frame, "Sent")
else:
    print["wrong frame"]
    print["NACK Sent"]
```

Output:

Sender:

Enter message: hello

Enter window size: 3

sender Buffer

Frame 1: h

Frame 2: e

Frame 3: l

Enter ACK received: 3

wrong ACK - Resending frames

sender Buffer:

Frame 1: h

Frame 2: e

Frame 3: l

Enter ACK received: 3

ACK accepted

Sender Buffer:

Frame 4: l

Frame 5: o

Sit frame successfully

Receiver:

Enter received frame no: 1

Enter frame data: h

Enter expected frame no: 1

Receive:

frame 1 received

Data: h

Ack 1 sent.

RESULT:

Thus the sliding window created was successfully implemented.

18/9

Ex:2
Date: 19/09

NMAP to Discover Live Hosts using Nmap Scans (ARP, ICMP, TCP/UDP) on the TryHackMe Platform

AIM:

To discover live hosts using NMAP scans ARP, ICMP, TCP/UDP) on the TRYHACKME Platform.

PROCEDURE:

1. Setup and Environment Initialization:

* Log in to the TryHackME platform and navigate to the target room: TRYHACKME NMap Room

* Display the target machine and establish a VPN connection using OpenVPN, or launch the browser-based AttackBox

2. ARP Scan for local Host Discovery

* Perform an ARP request scan to identify active hosts on the local network segment.

command:

Sudo nmap -PR -sn <target-IP(subnet)>

3. ICMP Echo Ping Scan:

* Send ICMP Request packets to check for TCMP Echo reply from active hosts.

command:

sudo nmap -PE -sn <target-IP(subnet)>

4: ICMP Timestamp Ping Scan:

Send ICMP Timestamp Request packet

to discover hosts that block standard ICMP Echo Requests.

Cmd:

Sudo nmap -PR -sn <target.IP/subnet>

5. ICMP address Mask Ping Scan.

* Send ICMP address Mask Request packets to identify live systems by requesting subnet mask information

Cmd:

Sudo nmap -PM -sn <Target-IP/subnet>

6. TCP SYN Ping Scan:

* Send TCP SYN packets (typically to on specified ports) to discover systems based on SYN-ACK (or) RST responses

Cmd:

Sudo nmap -PS -sn <target.IP/subnet>

7. TCP ACK Ping Scan:

*Send TCP ACK packets to elicit RST responses from active hosts

Cmd:

Sudo nmap -PA -sn <target:IP/subnet>

8. UDP Ping Scan

* Send empty UDP packets to destination ports to detect hosts via ICMP port unreachable responses.

Cmd:

Sudo nmap -PU -sn <target-IP/subnet>

9. Disable Port Scanning (Host Discovery only)

* Use the -sn flag with nmap to perform host discovery exclusively without triggering subsequent port scans.

Cmd:

Sudo nmap -sn <target-IP/subnet>

RESULT:

Thus, Live host discovery using various Nmap scanning techniques (ARP, ICMP Echo/Timestamp/Address Masks TCP SYN/ACK, and UDP pings) was successfully performed on the TryHackMe Platform.

19/9/24

Ex:9.
Date: 22/09

IMPLEMENTATION OF SUBNETTING IN CISCO PACKET TRACER SIMULATOR.

AIM:

To design, configure, and simulate a subnetted network topology in cisco Packet Tracer assign static IP addresses using classless IP subnetting across multiple subnets and observe packet transmission across the subnets with and without a router.

PROCEDURE:

1. Design Topology: open Cisco Packet Tracer and place Routers, Switches and PCs on the workspace.

2. Connect Cables: Connect PCs to switches and switches to Router interface using straight through cables.

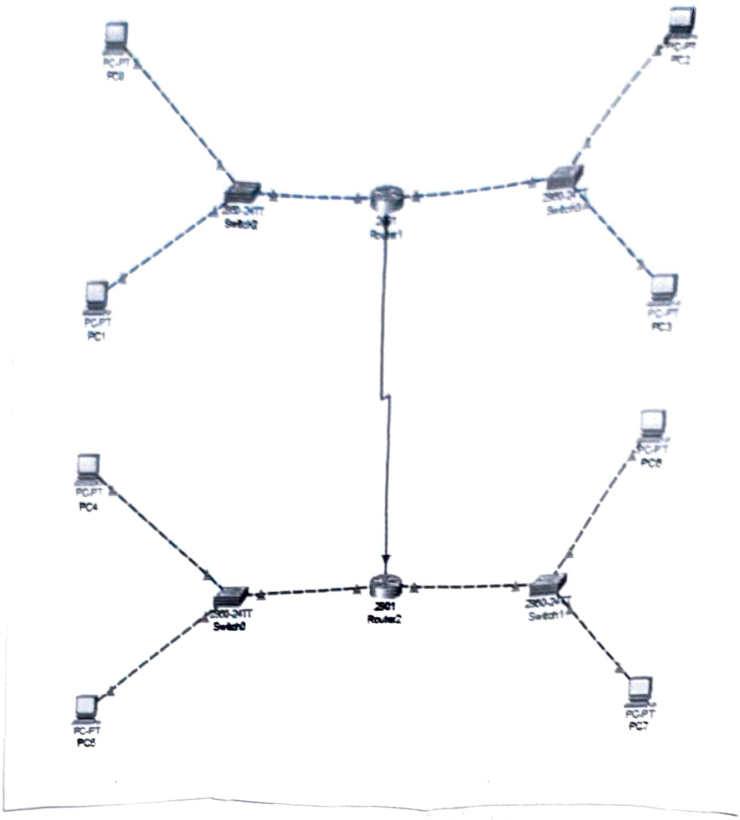
3. Assign IP address to PCs: Click Desktop IP configuration each PC and assign.

* Subnet Mask: 255.255.255.224 (/27)
* IPs and Gateway:

Subnet 1: PC IPs: 192.168.1.1 to 192.168.1.15 (Default Gateway = 192.168

Subnet 2: PC IPs = 192.168.2.1 to 192.168.2.15 (Default Gateway = 192.168.2.

4. Configure Router: Open Router CLI and configure interface



5. Test Communication:

* With Router: Ping a PC in subnet 2 from subnet 1 using command Prompt (ping 192.168.2.11) Ping will be successfully.

* Without Router: Disconnect the router and ping again. Ping will fail due to lack of inter-subnet routing.

RESULT

The subnetted network topology was successfully designed and configured in Cisco Packet Tracer.

22/9/24

END - END Communication at Transport Layer

A) Implement echo client server using TCP/UDP sockets.

AIM:

To write the program to implement echo client server using TCP/UDP sockets.

Code:

server.py

```
import socket
s = socket.socket()
s.bind(('127.0.0.1', 65432))
s.listen(1)
print("server ready...")
conn, addr = s.accept()
msg = conn.recv(1024)
print("Received:", msg.decode())
conn.send(msg)
conn.close()
s.close()
```

client.py

```
import socket
s = connect.socket()
s.connect(('127.0.0.1', 65432))
s.send(b"Hello server")
reply = s.recv(1024)
print("Echo from server:", reply.decode())
s.close()
```

RESULT :

Thus the above program is executed succesfully.

8) Implement chat client server using TCP/UDP sockets.

AIM:

To write the program to implement chat client server using TCP/UDP sockets.

Code:

data.server.py

```
import socket
S=socket.socket()
S.bind(('127.0.0.1', 65432))
S.listen(1)
print("waiting for client...")
conn, addr= S.accept()
while True:
    msg= conn.recv(1024).decode()
    if msg.lower()='bye':
        break
    print("client:", msg)
    reply= input('server:')
    conn.send[reply.encode()]
if reply.lower()=="bye".
    break
conn.close()
S.close()
```

Chat client.py

```
import socket
s = socket.socket()
s.connect(('127.0.0.1', 65432))
while True:
    msg = input('client :')
    s.send(msg.encode())
if msg.lower() == 'bye':
    break
reply = s.recv(1024).decode()
if reply.lower() == 'bye':
    break
print("server :", reply)
s.close()
```

Result:

Thus, the above program is executed successfully.

25/9/24

Ex: 12
Date: 26/6/24

Implement your own Ping Request

AIM:

To write the program to implement your own Ping Program.

Code:

server.py

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("127.0.0.1", 12000))
print("server is listening....")
while True
    data, address = server.recvfrom(1024)
print("Received:", data.decode())
server.sendto(data, address)
```

client.py

```
import socket
import time
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.settimeout(1)
for i in range(1,6):
    start = time.time()
```

```
client.sendto(b"ping{i}",encode(),
("127.0.0.1",12000))

try:
    data,_ = client.recvfrom(1024)
    rtt = (time.time() - start) * 1000
    print(f"reply: {data.decode()} Time=
{rtt:.2f}ms")
except socket.timeout
    print("Request timed out")
client.close()
```

26/9/24

RESULT

The above code is executed successfully.

Ex: 14

Date: 29/04

Write a code : using RAW sockets to implement packet sniffing.

AIM:

To write a code using RAW socket to implement packet sniffing

Code:

Sniffer.py

```
import socket
import .
```

```
Host = socket.gethostbyname(socket.gethostname())
```

```
Sock = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
Sock.bind((HOST,0))
Sock.setsockopt(socket.IPPROTO_IP,socket.IP_HDRINCL, 1)
Sock.ioctl(socket.SIO_RCVALL,socket.RCVALL_ON)
print("packet sniffing started...")
print("Host IP:", HOST)
print("Press Ctrl+C to stop.\n")
try:
    while true:
packet, address = sock.recvfrom(65565)
```

```
ip_header = packet[:20]
iph = struct.unpack("!BBHHHBBH4s4s", ip_header)
lower_ip = socket.inet_ntoa(iph[8])
destination_ip = socket.inet_ntoa(iph[9])
protocol = iph[6]
print("-----")
print("Source IP : "source_ip)
print("Destination IP :", destination_ip)
print("Protocol :", protocol)
print("Packet size :", len(mac))
print("-----")
except KeyboardInterrupt:
    print("\n Sniffing stopped.")
finally:
    sock.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
    sock.close()
```

RESULT:

29/9/25

The above code executed successfully

Ex.10 b)
Date: 30/09

Design and configure an internetwork using wireless source, DHCP server and internet cloud.

AIM:

To design and configure an internetwork using a wireless source, DHCP server and internet cloud in CISCO packet tracer.

PROCEDURE

1. Open CISCO packet tracer and create the required network topology
2. Connect PC, laptop, wireless source
3. Configure the wireless source with home network and enable DHCP
4. Connect the laptop to the wireless network and enable DHCP and DM.
5. Configure the server with IP 205.67.22 and enable DHCP and DM
6. Verify connectivity using IP config and ping CISCO con.

RESULT

The internetwork was successfully designed and configured using a source, DHCP server and internet cloud and connectivity was succes

Ex: 10
a)
Interworking with routers using Cisco packet tracer Simulator

AIM:
To design and configure a simple inter network using a source in Cisco packet tracer and fast connectivity b/w two PCs

PROCEDURE

Step 1 - configure source
* swap the routes and open CLI
* ensure to start configuring rates.
* type enable to activate the previous.

Step 2
* assign IP address to every PC on the network
* start the PC, go the desktop & select IP config, and assign and IP address
* design the default gateways of PC as. 160.10.1

Step 3
* connect fast port of PC0 with fast ethernet 8 port of server using a copper straight through cable

30

Ex:11(a) Simulate static routing configuration using
Date: 30/09 cisco packet tracer

AIM:

To simulate a static routing configuration using cisco packet tracer

PROCEDURE

* Open cisco packet tracer and create the given network topology
* connect 3 routers and PC's suitable
* configure IP address for router and PC
* enable all router interface using no shut down
* Configure static router on source IP, router1, router 2 using IP
* If main and backup router using different administrative distance
* test connectivities.
* use tracer to verify the routing path & disconnects.
* static routing is successfully compiled and verified

30

Ex: No: 11b)

Date: 30/04

Simulate using Cisco packet tracer

AIM:

To simulate RIP using CISCO packet tracer

PROCEDURE

1. Create the topology with 3 router & 2 pcs
2. Connect the device using suitable cable
3. assign the given IP address to both PC
4. Configure fast ethernet & serial interface of all routers.
5. Enables, interface using router RIP
6. Add directly connected network using the network command
7. Test connectivity using ping 20.0.0.2
8. Use traces 20.0.0.2. verify the suited routers.
9. Disconnect the direct nodes.

RESULT

Thus Static routing configuration RIP simulated using Cisco packet tracer